CS 4604 DBMS Connection Demo

Huayu Liang, GTA
10/15/2026

Contents

- Tech Stack
- Overview
- Demo Workflow

Tech Stack for the Demo

- Frontend: **React JS**
- Backend: **Flask** Rest API using Python
- Database: **MySQL**
- DBMS visualization tool: **phpMyAdmin**
- Operating system: MacOS
  - If you use a different OS; CLI may be different
- GitHub Repository for the demo: https://github.com/huayu1998/CS4604-DBMS-Demo.git

Overview

- Install and create a database
- Connect the DB to the application
- Execute SQL statements & Real-time date updates
- GUI tool to visualize and handling the data
- Collaboration among the team

Demo Workflow

1. Create the Flask API (backend)
2. Create the React app (frontend)
3. Install MySQL locally
4. Create a DB and populate sample data (STUDENT table) & Run CRUD via the SQL CLIs
5. Connect Flask (backend) ↔ MySQL (DB)
6. Connect React (frontend) ↔ Flask (backend)
7. Setup phpMyAdmin
8. Share the MySQL DB server (cloud)

Step 0 - Prerequisites

1. Install the Node.js and Python
2. Create a folder for the project

Step 1 - Backend (Flask API) Setup

1. Create a backend folder and venv & Activate the venv
2. Install the Flask package
3. Create a script and run to test

```python
from flask import Flask, jsonify

app = Flask(__name__)

@app.route("/")
def hello_world():
    return "Backend is running successfully :)\n"

@app.route("/api/students")
def students():
    return jsonify([{"studentID": 1, "name": "Alice", "major": "CS", "gpa": 3.8}])

if __name__ == "__main__":
    app.run(port=5001, debug=True)
```

Step 2 - Create the React App (Frontend)

1. Create a React app
2. Install dependencies and run
3. Open the localhost on default port

```
> npx
> 'create-vite' frontend --template react

|
◆  Which linter to use?
|  ● Oxlint
|  ○ ESLint
|  ↑/↓ to navigate • Enter: confirm
└
```

Step 3 - Install MySQL locally

1. Install the MySQL
2. Start the MySQL

Step 4 - Create a DB and Operate the Data

(STUDENT: studentID, name, major, gpa)

1. Create and populate a sample table
2. CRUD operations
    a. CREATE
    b. READ
    c. UPDATE
    d. DELETE
3. Type exit to leave the MySQL prompt

Step 5 - Connect Flask (backend) ↔ MySQL (DB)

1. Install the MySQL connector and dotenv (inside the backend/)
2. Store credentials in the backend/.env (never commit it!)
3. Update the app.py script with the DB-backed version
4. Run and test each endpoint (CRUD) with curl
5. Confirm the MySQL DB updates after the backend calls

Step 6 - Connect React (frontend) ↔ Flask (backend)

1. Update the frontend/src/App.jsx to connect React to Flask
2. Run frontend and backend
3. Execute and verify the data updates

```
const API = "http://127.0.0.1:5001/api/students";

fetch(API).then(r => r.json()).then(setStudents);                    // Read
fetch(API, { method: "POST", headers, body });                       // Create
fetch(`${API}/${id}`, { method: "PUT", headers, body });             // Update
fetch(`${API}/${id}`, { method: "DELETE" });                         // Delete
```

Step 7 - phpMyAdmin Setup

1. Prerequisite: install the Docker Desktop App
2. Run phpMyAdmin
3. Open http://localhost:8080 and log in with server, root and password
4. Execute and update the data & verify the change

Step 8 - Share the MySQL DB server

- Request a cloud MySQL server (e.g., Azure, Aiven, etc.)
- Connect the cloud server with the admin user
- Get the credentials (e.g., host, port, user, name, password, etc)
- Share the credentials among the team members (privately)
- Project setup via GitHub repository

Quick Docs Ref

- Prereq
  - Node.js Install & Download
  - Python venv
- Backend & Frontend
  - Flask: Quick Started & Flask: CORS
  - Vite: Getting Started
  - React: Start a New Project & Quick Start
- MySQL
  - Install MySQL on macOS & Install with Homebrew
  - Data Manipulation Statements (MySQL)
- MySQL-Flask Connect
  - Connect with Connector
  - Python-dotnev
  - Flask: HTTP methods
- React-Flask Connect
  - MDN: Using Fetch
  - React Doc
- PhpMyAdmin
  - Docker Desktop for Mac
  - phpMyAdmin Docker image

Resources

- **DBMS Servers** (Relational)
  - PostgreSQL: https://www.postgresql.org/
  - SQLite: https://www.sqlite.org/
  - Microsoft SQL Server: https://www.microsoft.com/en-us/sql-server
- **DBMS Visualization Tools**
  - DBeaver: https://dbeaver.io/
  - MySQL Workbench: https://www.mysql.com/products/workbench/
  - pgAdmin: https://www.pgadmin.org/
  - TablePlus: tableplus.com
- **Cloud MySQL Servers**
  - Aiven: https://aiven.io/free-mysql-database
  - AZure: https://azure.microsoft.com/en-us/pricing/purchase-options/azure-account?icid=mysql